

## میدلویرها، به زبان ساده

CHABOKPRO / BACKEND FIELD GUIDE

# از ورود درخواست تا پاسخ نهایی

راهنمای فارسی برای برنامه‌نویس جونیور  
همهٔ میدلویرهای اختصاصی، سرویس‌های وابسته،  
درخواست‌های تستی و پاسخ‌های قابل انتظار

## کد واقعی

مبنای تمام توضیحات

## ۵

سیاست محدودیت درخواست

## ۱۰

میدلویر اختصاصی

مثال‌ها آموزشی و مبتنی بر بررسی کد هستند. هیچ درخواست تغییردهنده‌ای روی سرور واقعی اجرا نشده است.

نسخه بررسی: ۲۰ سپتامبر ۲۰۲۶

نسخه Laravel در composer.lock: 13.22.0

## قبل از شروع: چه چیزی را می‌خوانیم؟

میدلویر مثل نگهبان یک مرحله از مسیر درخواست است. می‌تواند اطلاعاتی به درخواست اضافه کند، اجازه عبور بدهد یا همان‌جا پاسخ خطا برگرداند. کنترلر بعد از این نگهبان‌ها کار اصلی مثل گرفتن لیست نشست‌ها را انجام می‌دهد.

```
Client -> Middleware -> Middleware -> Controller
Client <- Response <- Response <- Controller
```

عبارت `$next($request)` یعنی «درخواست را به مرحله بعد بده و پاسخ آن را برای من برگردان». کدی که قبل از آن است روی ورودی کار می‌کند؛ کدی که بعد از آن است روی پاسخ کار می‌کند. اگر خطا یا پاسخ زودهنگام برگردد، مسیر عادی ادامه پیدا نمی‌کند.

| اصطلاح                    | معنی ساده                                               |
|---------------------------|---------------------------------------------------------|
| <b>Request / Response</b> | درخواست کلاینت / پاسخ سرور                              |
| <b>Header / Body</b>      | اطلاعات کنار درخواست / محتوای اصلی آن                   |
| <b>Principal</b>          | هویت تأییدشده: کاربر، نشست، سازمان و وضعیت رمز          |
| <b>hq_id یا Tenant</b>    | سازمان صاحب داده؛ از هویت معتبر به دست می‌آید           |
| <b>Node</b>               | شعبه یا گره عملیاتی انتخاب‌شده                          |
| <b>Contract / Adapter</b> | قرارداد یک کار / کلاسی که آن کار را واقعاً انجام می‌دهد |
| <b>Repository</b>         | بخشی که داده را از دیتابیس می‌خواند یا در آن می‌نویسد   |
| <b>Scope / Permission</b> | محدوده مجاز / کاری که اجازه انجامش را داریم             |

در این سند «سرویس‌های داخل میدلویر» شامل وابستگی‌های تزریق‌شده و زنجیره‌ای است که واقعاً صدا می‌زنند. همه سرویس‌های تجاری پروژه مثل قیمت‌گذاری، موضوع این راهنما نیستند.

## قرارداد مثال‌ها و فهرست مطالعه

آدرس `api.example.test` فرضی است. برای تست دستی آن را با آدرس محیط توسعه جایگزین کن. همه نمونه‌ها HTTPS فرض شده‌اند، مگر مثالی که صریحاً HTTP را بررسی می‌کند. `TOKEN_VALID` و `REFRESH_VALID` باید با توکن واقعی محیط تست جایگزین شوند.

```
Origin allowed: https://panel.example.test
Correlation ID: 11111111-1111-4111-8111-111111111111
Node ID:      22222222-2222-4222-8222-222222222222
User ID:      33333333-3333-4333-8333-333333333333
Session ID:   44444444-4444-4444-8444-444444444444
Tenant ID:    55555555-5555-4555-8555-555555555555
```

فرض پایه: کاربر و نشست فعال‌اند، توکن منقضی نیست، رمز اجباری تغییر کرده و هر جا لازم باشد مجوز و محدوده مناسب وجود دارد. شناسه‌ها و زمان‌ها داده فرضی هستند؛ ساختار پاسخ‌ها از کد برداشته شده است.

| بخش                                      | صفحه      |
|------------------------------------------|-----------|
| نقشه اجرا و قالب پاسخ                    | ۴ تا ۶    |
| میدلویرهای عمومی API                     | ۷ تا ۱۱   |
| احراز هویت و سرویس‌های توکن و نشست       | ۱۲ تا ۱۴  |
| دسترسی شعبه، سرویس‌های مجوز و Audit      | ۱۵ تا ۱۷  |
| تغییر رمز و خروج از همه نشست‌ها          | ۱۸ تا ۲۰  |
| درخواست‌های تکراری و سرویس‌های آن        | ۲۱ تا ۲۴  |
| محدودکننده درخواست و لایه Laravel        | ۲۵ تا ۲۷  |
| سناریوی کامل، تست دستی، عیب‌یابی و منابع | ۲۸ به بعد |

مسیرهای `/api/v1/foundation/` فقط در `ApiFoundationTest` ثبت می‌شوند و API عمومی پروژه نیستند. این مثال‌ها برای جدا کردن رفتار میدلویر از منطق تجاری انتخاب شده‌اند. مسیرهای دیگر از فایل‌های `Routes` واقعی هستند.

## نقشه اجرا: کدام نگهبان اول می‌آید؟

در bootstrap/app.php چهار میدلویر به ابتدای گروه api اضافه شده‌اند. این‌ها میدلویر سراسری کل برنامه نیستند؛ مثلاً مسیر سلامت up/ عضو این گروه نشده است. لایه عمومی Laravel قبل از گروه مسیر اجرا می‌شود.

```
Laravel global middleware
-> CorrelationId
-> RequireSecureTransport
-> CredentialedCors
-> NodeContext
-> SubstituteBindings (framework)
-> route middleware
-> FormRequest / Controller / Use case
```

| alias یا نام           | وظیفه                            |
|------------------------|----------------------------------|
| CorrelationId          | شماره پیگیری و ثبت زمان پاسخ     |
| RequireSecureTransport | اجبار ارتباط امن                 |
| CredentialedCors       | هدرهای دسترسی مرورگر             |
| NodeContext            | خواندن و بررسی قالب X-Node-Id    |
| access.auth            | توکن + اعتبار نشست               |
| node.access            | مجوز انتخاب شعبه + Audit         |
| password.changed       | محدود کردن کاربر با رمز موقت     |
| origin.exact           | بررسی دقیق مبدأ درخواست          |
| idempotent:command     | جلوگیری از اجرای تکراری یک فرمان |
| AuthenticateLogoutAll  | احراز هویت ویژه خروج همگانی      |

این نقشه ترتیب ثبت را نشان می‌دهد. Laravel برای برخی میدلویرهای خودش اولویت داخلی دارد. همچنین پاسخ preflight ممکن است در HandleCors سراسری ساخته شود و اصلاً به گروه api نرسد.

مبنای کد:

```
bootstrap/app.php
Modules/Identity/src/Presentation/Routes/api.php
```

## روی کدام مسیرها اجرا می‌شوند؟

تمام گروه‌های زیر api دارند؛ چهار میدلویر ابتدایی در همه آن‌ها مشترک‌اند. alias فقط نام کوتاه یک کلاس است؛ ثبت alias به‌تنهایی آن را روی همه مسیرها فعال نمی‌کند.

| مسیر / خانواده                                                                                                 | میدلویر اضافه                                                         |
|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| POST /auth/login                                                                                               | throttle:auth-login                                                   |
| POST /auth/refresh                                                                                             | origin.exact + throttle:auth-refresh                                  |
| POST /auth/logout                                                                                              | origin.exact                                                          |
| POST /auth/logout-all                                                                                          | origin.exact + AuthenticateLogoutAll                                  |
| POST /auth/otp/send و verify/                                                                                  | throttle:auth-otp-send / auth-otp-verify                              |
| POST /auth/password/activate و reset/                                                                          | throttle:auth-password                                                |
| POST /auth/password/change                                                                                     | access.auth + node.access<br>password.changed حذف شده                 |
| DELETE /me/sessions/{id} و GET /me/sessions                                                                    | access.auth + node.access +<br>password.changed                       |
| User, Authorization, Dashboard, Manifest consignments<br>عملیات اجرایی و                                       | access.auth + node.access +<br>password.changed                       |
| Geography, Organization, ServiceCatalog, Pricing, Fleet, Network operational-statuses و شماره بارنامه و config | access.auth + password.changed<br>node.access در این گروه‌ها ثبت نشده |

پیشوند مسیرهای بالا /api/v1 است. idempotent فقط روی دستورهای که در Routes با آن علامت خورده‌اند اجرا می‌شود؛ مثلاً ساخت نقش، ساخت کاربر، ساخت بارنامه، عملیات مشخص مانیفست، قیمت‌گیری و برخی عملیات ناوگان/شبکه. هر POST الزاماً idempotent نیست.

نمود `node.access` به معنی نبود کنترل دسترسی تجاری نیست. Use case می‌تواند مجوز جداگانه بررسی کند. `NodeContext` همچنان قالب هدر شعبه را برای تمام گروه api بررسی می‌کند.

مبنای کد:

Modules/\*/src/Presentation/Routes/\*.php  
bootstrap/app.php

## پاسخ نهایی چطور ساخته می‌شود؟

### ApiResponse: بسته‌بندی یکدست پاسخ

متد success داده کنترلر را در data می‌گذارد. meta اطلاعات جانبی مثل صفحه‌بندی است. هر دو متد SUCCESS و error شناسه پیگیری را در بدنه و هدر می‌گذارند. آبجکت خالی در JSON به شکل {} است.

```
HTTP 200
Content-Type: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111

{
  "data": {
    "success": true
  },
  "meta": {},
  "correlation_id": "11111111-1111-4111-8111-111111111111"
}
```

### ApiExceptionRenderer: تبدیل خطای PHP به پاسخ HTTP

این کلاس میدلویر نیست؛ در bootstrap به مدیریت خطا متصل شده است. فقط `*/api` را پوشش می‌دهد. `ApiException` کد، وضعیت و پیام را تعیین می‌کند؛ خطای اعتبارسنجی 422 و خطای ناشناخته 500 می‌شود.

```
HTTP 401
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Content-Type: application/json

{
  "error_code": "AUTHENTICATION_REQUIRED",
  "message": "Authentication required.",
  "field_errors": {},
  "details": {},
  "correlation_id": "11111111-1111-4111-8111-111111111111"
}
```

ApiResponse برای Origin مجاز، هدر CORS را روی پاسخ خطا هم می‌گذارد؛ بنابراین فقط عبور عادی از CredentialedCors مسئول این هدرها نیست. جزئیات خام خطای غیرمنتظره به کلاینت داده نمی‌شود.

مبنای کد:

Modules/Foundation/src/Presentation/Http/ApiResponse.php

Modules/Foundation/src/Presentation/Http/ApiExceptionRenderer.php

## ۱. CorrelationId: شماره پیگیری درخواست

مثل شماره رسید است: با یک شناسه، پاسخ کلاینت و لاگ سرور را به هم وصل می‌کنیم. اگر X-Correlation-ID خالی باشد UUID جدید می‌سازد؛ اگر آمده باشد باید UUID باشد. فاصله ابتدا و انتها حذف می‌شود.

- شناسه در request.attributes با کلید correlation\_id قرار می‌گیرد.
- Log::shareContext شناسه، متد و مسیر را به زمینه مشترک لاگ اضافه می‌کند.
- پس از دریافت پاسخ، X-Correlation-ID را تنظیم و http.request.completed را با status و duration\_ms ثبت می‌کند.
- finally زمینه مشترک لاگ را پاک می‌کند. سرویس اختصاصی تزریق شده ندارد؛ از Str, Log و ApiResponse استفاده می‌کند.

### درخواست تستی در ApiFoundationTest

```
GET /api/v1/foundation/success HTTP/1.1
Host: api.example.test
Accept: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
```

```
HTTP 200
Content-Type: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111

{
  "data": {
    "ok": true
  },
  "meta": {},
  "correlation_id": "11111111-1111-4111-8111-111111111111"
}
```

### ورودی خراب

همین درخواست را با X-Correlation-ID: bad-id بفرست. پاسخ 422 است و field\_errors نام هدر را دارد. کد فعلی همان مقدار نامعتبر را در correlation\_id و هدر پاسخ تکرار می‌کند؛ آن را خودکار اصلاح نمی‌کند.

```
error_code: VALIDATION_ERROR
message: The request is invalid.
field_errors.X-Correlation-ID[0]:
  The X-Correlation-ID field must be a UUID.
correlation_id: bad-id
```

مبنای کد:

Modules/Foundation/src/Presentation/Http/Middleware/CorrelationId.php  
Modules/Foundation/tests/Feature/ApiFoundationTest.php

## ۲. RequireSecureTransport: ارتباط باید امن باشد

این نگهبان از `request.isSecure` می‌پرسد ارتباط HTTPS است یا نه. اگر نیست، فقط در `local`، `testing` یا `staging` و با فعال بودن `local_http_allowed` اجازه عبور می‌دهد. `production` حتی با این پرچم اجازه HTTP نمی‌دهد.

| محیط / شرط                                                          | نتیجه |
|---------------------------------------------------------------------|-------|
| HTTPS                                                               | عبور  |
| HTTP در <code>local/testing/staging</code> + پرچم <code>true</code> | عبور  |
| HTTP در همان محیط‌ها + پرچم <code>false</code>                      | 403   |
| HTTP در <code>production</code>                                     | 403   |

### درخواست تستی با HTTP و استثنای خاموش

```
GET /api/v1/foundation/success HTTP/1.1
Host: api.example.test
Accept: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
```

URL کامل این درخواست: `http://api.example.test/api/v1/foundation/success`

```
HTTP 403
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Content-Type: application/json

{
  "error_code": "FORBIDDEN",
  "message": "Secure transport is required.",
  "field_errors": {},
  "details": {},
  "correlation_id": "11111111-1111-4111-8111-111111111111"
}
```

همین درخواست با `https` و شرایط سالم به پاسخ 200 با `data: {"ok": true}` صفحه قبل می‌رسد. میدلویر `redirect` به HTTPS انجام نمی‌دهد؛ درخواست ناامن را رد می‌کند.

### سرویس و تنظیمات

سرویس اختصاصی ندارد. `app()->environment` محیط و مقدار `LOCAL_HTTP_ALLOWED` را می‌دهد. پشت `reverse proxy`، تشخیص `isSecure` به تنظیم اعتماد به پراکسی وابسته است. `bootstrap` این پروژه `trustProxies` سفارشی ندارد؛ صرف فرستادن `X-Forwarded-Proto` را معادل اثبات HTTPS ندان.

مبنای کد:

Modules/Foundation/src/Presentation/Http/Middleware/RequireSecureTransport.php  
config/chabok.php

## ۳. CredentialedCors: مرورگر اجازه خواندن دارد؟

وقتی پنل و API روی Origin متفاوت باشند، مرورگر برای خواندن پاسخ به هدرهای CORS نگاه می‌کند. Origin ترکیب scheme، host و port است. این کلاس فقط Origin دقیق موجود در branch\_panel.origins را مجاز می‌داند و credentials را فعال می‌کند.

- در درخواست معمولی ابتدا next اجرا می‌شود؛ بعد Vary: Origin و در صورت مجاز بودن مبدأ، هدرهای CORS اضافه می‌شوند.
- درخواست معمولی با مبدأ نامجاز توسط این کلاس به‌تنهایی متوقف نمی‌شود. معمولاً مرورگر اجازه خواندن پاسخ را نمی‌دهد.
- در خود این کلاس، OPTIONS دارای Access-Control-Request-Method در صورت Origin مجاز 204 خالی و در غیر این صورت 403 می‌دهد. این رفتار را از رفتار کل برنامه جدا کن.

### تست درخواست معمولی با مبدأ مجاز

```
GET /api/v1/foundation/success HTTP/1.1
Host: api.example.test
Accept: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Origin: https://panel.example.test
```

```
HTTP 200
Content-Type: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Access-Control-Allow-Origin: https://panel.example.test
Access-Control-Allow-Credentials: true
Access-Control-Expose-Headers: X-Correlation-ID
Vary: Origin

{
  "data": {
    "ok": true
  },
  "meta": {},
  "correlation_id": "11111111-1111-4111-8111-111111111111"
}
```

هدرهای مجاز: GET، Authorization، Content-Type، X-Correlation-ID، X-Node-Id، Idempotency-Key، متدهای مجاز: POST، PATCH، PUT، DELETE، OPTIONS. پاسخ معمولی این کلاس Allow-Headers و Allow-Methods را نیز می‌گذارد؛ بلا فقط هدرهای کلیدی نمایش داده شده‌اند.

مبنای کد:

```
Modules/Foundation/src/Presentation/Http/Middleware/CredentialedCors.php
config/cors.php
config/chabok.php
```

## ۴. RequireExactOrigin: بررسی اجباری مبدأ

این کلاس برخلاف کنترل خواندن پاسخ در مرورگر، واقعاً جلوی ادامه درخواست را می‌گیرد. در refresh, logout و logout-all استفاده شده است؛ این مسیرها با کوکی سروکار دارند. Origin خالی یا خارج از فهرست دقیق، رد می‌شود.

### درخواست واقعی با Origin اشتباه

```
POST /api/v1/auth/logout HTTP/1.1
Host: api.example.test
Accept: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Origin: https://evil.example.test
Content-Type: application/json

{}
```

```
HTTP 403
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Content-Type: application/json

{
  "error_code": "ORIGIN_NOT_ALLOWED",
  "message": "Origin is not allowed.",
  "field_errors": {},
  "details": {},
  "correlation_id": "11111111-1111-4111-8111-111111111111"
}
```

### درخواست صحیح و پاسخ موفق

Origin را به <https://panel.example.test> که در فهرست مجاز است تغییر بده. حتی بدون کوکی، logout موفق است؛ LogoutHandler در نبود توکن کاری نمی‌کند و کنترلر کوکی را پاک می‌کند.

```
HTTP 200
Content-Type: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111

{
  "data": {
    "success": true
  },
  "meta": {},
  "correlation_id": "11111111-1111-4111-8111-111111111111"
}
```

## سرویس SecurityMetricRecorder

پیاده‌سازی RedisSecurityMetricRecorder قبل از رد کردن، شمارنده auth.origin\_rejected را زیاد می‌کند. label reason برای نبود Origin برابر MISSING و برای مبدأ نامجاز UNLISTED است. این سرویس Redis connection(cache) و HINCRBY را استفاده می‌کند؛ در این کلاس تعیین نشده. متد snapshot شمارنده‌ها را به عدد صحیح برمی‌گرداند.

```
Redis hash: chabok:security:metric:auth.origin_rejected
Field: reason=UNLISTED
Change: +1
```

مبنای کد:

Modules/Foundation/src/Presentation/Http/Middleware/RequireExactOrigin.php

Modules/Foundation/src/Infrastructure/Observability/RedisSecurityMetricRecorder.php

## ۵. NodeContext: فقط شعبه انتخاب شده را بخوان

X-Node-Id می‌گوید کاربر می‌خواهد در کدام شعبه کار کند. این میدلویر فاصله‌ها را حذف می‌کند؛ مقدار خالی را null و UUID معتبر را در node\_id می‌گذارد. در این مرحله نه دیتابیس خوانده می‌شود و نه دسترسی تأیید می‌شود.

### درخواست تستی و خروجی کنترلر تست

```
GET /api/v1/foundation/context HTTP/1.1
Host: api.example.test
Accept: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
X-Node-Id: 22222222-2222-4222-8222-222222222222
```

```
HTTP 200
Content-Type: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111

{
  "data": {
    "node_id": "22222222-2222-4222-8222-222222222222",
    "tenant_id": null
  },
  "meta": {},
  "correlation_id": "11111111-1111-4111-8111-111111111111"
}
```

tenant\_id همچنان null است چون این مسیر تست فقط NodeContext را نشان می‌دهد و احراز هویت نشده است. سازمان از هدر شعبه استخراج نمی‌شود؛ AuthenticateAccessToken آن را از principal می‌گیرد.

### هدر خراب

همان درخواست با X-Node-Id: branch-1 پاسخ زیر را دارد:

```
HTTP 422
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Content-Type: application/json

{
  "error_code": "VALIDATION_ERROR",
  "message": "The request is invalid.",
  "field_errors": {
    "X-Node-Id": [
      "The X-Node-Id field must be a UUID."
    ]
  },
  "details": {},
  "correlation_id": "11111111-1111-4111-8111-111111111111"
}
```

مبنای کد:

Modules/Foundation/src/Presentation/Http/Middleware/NodeContext.php

## ۶. AuthenticateAccessToken: تو واقعاً چه کسی هستی؟

alias این کلاس access.auth است. ابتدا Authorization را با الگوی Bearer می‌خواند؛ بزرگی و کوچکی حروف Bearer مهم نیست. توکن خام به AccessTokenService و سپس claims آن به AccessSessionValidator داده می‌شود.

```
Authorization: Bearer TOKEN_VALID
-> tokens.decode(token)      : AccessTokenClaims
-> sessions.validate(claims) : AuthenticatedPrincipal
-> attributes.principal      = principal
-> attributes.tenant_id      = principal.hqId
-> next(request)
```

### درخواست واقعی بدون توکن

```
GET /api/v1/me/sessions HTTP/1.1
Host: api.example.test
Accept: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
```

```
HTTP 401
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Content-Type: application/json

{
  "error_code": "AUTHENTICATION_REQUIRED",
  "message": "Authentication required.",
  "field_errors": {},
  "details": {},
  "correlation_id": "11111111-1111-4111-8111-111111111111"
}
```

### حالت موفق

همان درخواست با Authorization: Bearer TOKEN\_VALID عبور می‌کند. اگر وضعیت رمز و شعبه هم اجازه بدهند، کنترلر لیست نشست‌ها را با 200 می‌دهد؛ نمونه کامل بدنه در صفحه ۲۸ آمده است. خود این میدلویر هیچ access\_token جدیدی صادر نمی‌کند.

سالم بودن امضای JWT کافی نیست. ممکن است کاربر غیرفعال شده باشد یا نشست او در دیتابیس یا Redis لغو شده باشد. همچنین این کد principal را در attributes ذخیره می‌کند و مستقیماً Auth::login یا request.user را تنظیم نمی‌کند.

منای کد:

Modules/Foundation/src/Presentation/Http/Middleware/AuthenticateAccessToken.php

## سرویس توکن: AccessTokenService

قرارداد AccessTokenService در Foundation به FirebaseAccessTokenService متصل است. نام Firebase در اینجا به کتابخانه JWT اشاره دارد؛ این کد درخواست HTTP به سرویس ابری Firebase نمی‌فرستد.

### issue: ساخت توکن پس از ورود

یک principal می‌گیرد، زمان فعلی را حساب می‌کند، شناسه تصادفی jti می‌سازد و با HS256 امضا می‌کند. TTL پیش‌فرض ۳۰۰ ثانیه و حداقل مقدار پذیرفته‌شده در issue برابر ۶۰ ثانیه است. خروجی token و expires\_in است. این متد در مسیر صدور نشست صدا زده می‌شود، نه در access.auth.

| معنی                                   | claim                |
|----------------------------------------|----------------------|
| صادرکننده و مصرف‌کننده مورد انتظار     | iss / aud            |
| شناسه کاربر و نشست                     | sub / sid            |
| سازمان؛ برای برخی کاربران پلتفرمی null | hq_id                |
| وضعیت رمز هنگام صدور توکن              | must_change_password |
| زمان صدور / زمان شروع اعتبار / انقضا   | iat / nbf / exp      |
| شناسه یکتای خود توکن                   | jti                  |

### decode: خواندن و بررسی توکن

JWT::decode با کلید HS256 امضا و قواعد زمانی کتابخانه را بررسی می‌کند. سپس این کلاس iss و aud را با تنظیمات برابر می‌خواهد و AccessTokenClaims می‌سازد. خطای decode به AUTHENTICATION\_REQUIRED با 401 تبدیل می‌شود.

```
CHABOK_JWT_ISSUER=chabok-api
CHABOK_JWT_AUDIENCE=chabok-branch-panel
CHABOK_ACCESS_TOKEN_TTL=300
CHABOK_JWT_SECRET=<at least 32 bytes>
```

secret از تنظیمات می‌آید؛ اگر کمتر از ۳۲ بایت باشد خطای پیکربندی دارد. در decode خطاهای غیر ApiException به 401 تبدیل می‌شوند؛ پس خرابی پیکربندی نیز ممکن است ظاهراً شبیه توکن نامعتبر دیده شود.

مبنای کد:

```
Modules/Foundation/src/Infrastructure/Security/FirebaseAccessTokenService.php
Modules/Foundation/src/Infrastructure/Providers/FoundationServiceProvider.php
config/chabok.php
```

## سرویس نشست: JWT معتبر هنوز کافی نیست

### AccessSessionValidator → DatabaseAccessSessionValidator

ورودی claims توکن و خروجی principal به روز است. ابتدا Redis را برای لغو نشست می خواند؛ سپس user\_sessions را با users join می کند. نشست نباید revoked باشد، باید هنوز تاریخ اعتبار داشته باشد، user\_id و hq\_id باید با توکن یکسان باشند و وضعیت کاربر ACTIVE باشد. در غیر این صورت 401 می دهد.

mustChangePassword را از دیتابیس می گیرد، نه از مقدار قدیمی توکن. بنابراین بعد از تغییر اجباری رمز، وضعیت جاری کاربر ملاک است. اگر hq\_id برابر null باشد، وجود نقش فعال پلتفرمی نیز بررسی می شود.

### RedisSessionRegistry: دفتر نشست های لغوشده

```
isInvalidated(sessionId)
-> EXISTS iam:session:revoked:{sessionId}
invalidate(sessionId, ttl=2592000)
-> SETEX key ttl "1"
```

در access.auth فقط isInvalidated صدا می شود. invalidate هنگام لغو نشست به کار می رود. دو متد rememberRotatedToken و rotatedFamily نیز هش refresh قبلی و خانواده توکن را برای گردش refresh نگه می دارند؛ آن ها در همین میدلویر اجرا نمی شوند.

### PlatformContextValidator: کاربر بدون سازمان

Adapter به AuthorizationService و سپس AuthorizationGuard و Repository می رود. باید کاربر فعال با hq\_id خالی، assignment فعال از نوع PLATFORM بدون scope\_id، و نقش فعال platform\_super\_admin داشته باشد. پیاده سازی پشتیبان UnavailablePlatformContextValidator همیشه false است؛ با فعال بودن مازول Authorization پیاده سازی واقعی جایگزین می شود.

هر درخواست احرازشده در این پیاده سازی دیتابیس را می خواند؛ استفاده از JWT به معنی حذف دیتابیس نیست. برای تست خطای زیرساخت، پاسخ را با «توکن منقضی» یکی فرض نکن؛ خطای Redis یا DB که مهار نشده باشد می تواند 500 شود.

مبنای کد:

Modules/Identity/src/Infrastructure/Security/DatabaseAccessSessionValidator.php

Modules/Identity/src/Infrastructure/Security/RedisSessionRegistry.php

Modules/Authorization/src/Infrastructure/Adapters/AuthorizationPlatformContextValidator.php

## ۴. ValidateNodeAccess: حق ورود به این شعبه را داری؟

alias آن node.access است و بعد از احراز هویت ثبت شده. فقط وقتی node\_id رشته باشد و principal از نوع AuthenticatedPrincipal باشد اجرا می‌شود. اگر هدر شعبه نداشته باشیم، از این کلاس عبور می‌کنیم؛ پس این میدلویر «الزامی بودن انتخاب شعبه» را بررسی نمی‌کند.

- NodeAccessValidator::assertAccessible مجوز و محدوده را بررسی می‌کند.
- پس از تأیید، AuditWriter رویداد NODE\_CONTEXT\_SELECTED را با targetType=NODE و sourceClient=BRANCH\_PANEL ثبت می‌کند.
- سپس next اجرا می‌شود؛ موفقیت این مرحله به معنی موفقیت قطعی کار تجاری نیست.

### درخواست واقعی با شعبه متعلق به سازمان دیگر

```
GET /api/v1/me/sessions HTTP/1.1
Host: api.example.test
Accept: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Authorization: Bearer TOKEN_VALID
X-Node-Id: 22222222-2222-4222-8222-222222222222
```

```
HTTP 403
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Content-Type: application/json

{
  "error_code": "TENANT_ACCESS_DENIED",
  "message": "Access denied.",
  "field_errors": {},
  "details": {},
  "correlation_id": "11111111-1111-4111-8111-111111111111"
}
```

### اگر شعبه مجاز باشد

با فرض وجود node\_context.switch و محدوده معتبر، Audit نوشته می‌شود و پاسخ نهایی همان لیست نشست‌ها در صفحه ۲۸ است. اگر مجوز نباشد PERMISSION\_DENIED، ماژول لازم فعال نباشد ENTITLEMENT\_DISABLED، یا شعبه خارج از محدوده باشد SCOPE\_ACCESS\_DENIED می‌گیری.

هر UUID با مقدار ناموجود هم ممکن است به خطای دسترسی برسد؛ تضمینی نیست که حتماً 404 برگردد. انتخاب شعبه به‌تنهایی tenant\_id را تغییر نمی‌دهد.

مبنای کد:

Modules/Foundation/src/Presentation/Http/Middleware/ValidateNodeAccess.php  
Modules/Authorization/src/Application/Services/AuthorizationGuard.php

## سرویس‌های بررسی دسترسی، یکی یکی

### ۱. NodeAccessValidator و Adapter آن

قرارداد فقط می‌گوید assertAccessible باید دسترسی را بررسی کند. AuthorizationNodeAccessValidator آن را به Foundation می‌سپارد. AuthorizationService::assertNodeAccessible ابتدا DenyNodeAccessValidator را ثبت می‌کند که همیشه 403 FORBIDDEN می‌دهد؛ Authorization با اولویت بالاتر adapter واقعی را ثبت می‌کند.

### ۲. AuthorizationGuard و AuthorizationService

AuthorizationService نمای ورودی ماژول است؛ در این مسیر کار اصلی را AuthorizationGuard انجام می‌دهد. ابتدا سازمان شعبه را می‌سنجد؛ سپس مجوز node\_context.switch و فعال بودن entitlement مربوطه؛ در پایان محدوده همان مجوز را می‌سنجد. مجوزهای یک نقش دیگر نباید خودکار محدوده این مجوز را باز کنند.

### ۳. ResolveContextHandler

سند دسترسی مؤثر کاربر را می‌سازد: نقش‌ها، مجوزها، permission\_scopes، منوها، شعب و ماژول‌های فعال. ابتدا کش را می‌خواند؛ اگر داده دارای permission\_scopes باشد برمی‌گرداند. در cache miss، کاربر فعال و سازمان را بررسی می‌کند و نقش‌ها و مجوزها را از Repository می‌خواند. نتیجه ۶۰ ثانیه کش می‌شود.

### ۴. AuthorizationRepository → EloquentAuthorizationRepository

خواندن users, roles, user\_role\_assignments, permissions, role\_permissions, nodes و tenant\_module\_entitlements را انجام می‌دهد. nodeTenant سازمان شعبه را می‌دهد؛ activePermission و entitlementStatus وضعیت مجوز و ماژول را تعیین می‌کنند؛ hasActivePlatformAssignment شرط دقیق کاربر پلتفرمی را می‌سنجد.

### ۵. AuthorizationCacheInvalidator و AuthorizationCache

RedisAuthorizationCache روی اتصال cache کار می‌کند: get خواندن، put با SETEX نوشتن و forget حذف است. Invalidator کلید chabok:authz:effective:{userId} را تولید و کش کاربر، کاربران سازمان یا کاربران نقش را پاک می‌کند. در مسیر خواندن، از آن برای ساخت کلید استفاده می‌شود.

مبنای کد:

Modules/Authorization/src/Infrastructure/Adapters/AuthorizationNodeAccessValidator.php

Modules/Authorization/src/Application/Services/AuthorizationGuard.php

Modules/Authorization/src/Application/UseCases/ResolveContext/ResolveContextHandler.php

## محاسبه محدوده و ثبت Audit

### ContextScopeResolver

برای context کلی، assignment های TENANT را به تمام شعب فعال سازمان، NODE را به شعبه مشخص و AREA را به شعب منطقه تبدیل می کند. includes\_descendants تعیین می کند زیرمنطقه ها نیز حساب شوند یا نه. Repository در پایان فقط شعب فعال سازمان را برمی گرداند.

### ScopeTopology و ScopedAccess

برای اجازه واقعی انتخاب شعبه، ScopedAccess::nodes فقط scope های node\_context.switch را از permission\_scopes برمی دارد. برای AREA، درخت زیرمنطقه ها را پیمایش می کند و مجموعه دیده شده جلوی تکرار را می گیرد. EloquentScopeTopology داده های درخت منطقه و شعب را از دیتابیس می خواند و hq\_id و activeOnly را اعمال می کند.

### RoleNavigationRepository و RoleNavigation

ResolveContext برای نمایش پنل، menu\_keys را هم حساب می کند. اگر هیچ نقشی نباشد [] و اگر بعضی نقش ها تنظیم صریح منو نداشته باشند null برمی گرداند؛ یعنی نمایش خودکار بر اساس مجوزها. EloquentRoleNavigationRepository تنظیم ها و آیتم های منو را می خواند. منوی قابل نمایش، به تنهایی مجوز API نیست.

### AuditWriter → MySqlAuditWriter

یک رکورد در audit\_events می نویسد: UUID رویداد، سازمان، انجام دهنده، نوع و شناسه هدف، correlation\_id، action و زمان. before/after و یادداشت در صورت وجود از SensitiveDataRedactor عبور می کنند. IP و User-Agent در صورت ارسال، هش می شوند؛ ValidateNodeAccess این دو را ارسال نمی کند.

```
action_key = NODE_CONTEXT_SELECTED
target_type = NODE
target_id = 22222222-2222-4222-8222-222222222222
source_client = BRANCH_PANEL
correlation_id = 11111111-1111-4111-8111-111111111111
```

SensitiveDataRedactor کلیدهایی مثل token، password و پسوندهای حساس را در آرایه ها با [REDACTED] جایگزین می کند و الگوهای حساس متن را نیز می پوشاند. Audit پس از مجوز نوشته می شود؛ شکست خود نوشتن Audit می تواند ادامه درخواست را متوقف کند.

مبنای کد:

Modules/Foundation/src/Application/ScopedAccess.php

Modules/Organization/src/Infrastructure/Adapters/EloquentScopeTopology.php

Modules/Audit/src/Infrastructure/Persistence/MySqlAuditWriter.php

## ۸. RequirePasswordChangeCompleted: اول رمزت را عوض کن

alias آن password.changed است. اگر principal معتبر باشد و mustChangePassword=true باشد، فقط نام مسیرهای موجود در forced\_password\_allowed\_routes اجازه عبور دارند. مقایسه بر اساس route name است؛ URL را با این فهرست مقایسه نمی‌کند.

```
Allowed route names:
auth.password.change
auth.refresh
auth.logout
auth.logout-all
```

### درخواست با نشست دارای رمز موقت

```
GET /api/v1/me/sessions HTTP/1.1
Host: api.example.test
Accept: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Authorization: Bearer TOKEN_MUST_CHANGE
```

```
HTTP 403
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Content-Type: application/json

{
  "error_code": "PASSWORD_CHANGE_REQUIRED",
  "message": "Password change is required.",
  "field_errors": {},
  "details": {},
  "correlation_id": "11111111-1111-4111-8111-111111111111"
}
```

### راه خروج از این وضعیت

در POST /api/v1/auth/password/change در Routes صریحاً withoutMiddleware("password.changed") دارد؛ ولی access.auth و node.access باقی‌اند. بعد از تغییر موفق رمز، همان GET با وضعیت تازه دیتابیس می‌تواند 200 بدهد.

```
POST /api/v1/auth/password/change
Authorization: Bearer TOKEN_MUST_CHANGE
Content-Type: application/json

{"current_password": "OldPassword!123",
 "new_password": "NewPassword!456"}
```

پاسخ موفق کنترلر: HTTP 200 با data برابر {"success": true}، meta برابر {} و correlation\_id درخواست. سرویس اختصاصی در این میدلویر تزریق نشده؛ principal و config را می‌خواند. اگر principal نباشد، خودش 401 نمی‌دهد؛ به access.auth قبل از خودش متکی است.

مبنای کد:

```
Modules/Foundation/src/Presentation/Http/Middleware/RequirePasswordChangeCompleted.php
Modules/Identity/src/Presentation/Routes/api.php
config/chabok.php
```

## ۹. AuthenticateLogoutAll: خروج حتی با Access Token منقضی

این میدلویر فقط روی POST /auth/logout-all و با نام کامل کلاس ثبت شده است. ابتدا تلاش می‌کند از Bearer هویت معتبر بسازد؛ اگر هدر وجود نداشته باشد یا decode/validate در Throwables بدهد، مسیر refresh cookie را امتحان می‌کند.

```
Access token valid?
yes -> principal
no -> read chabok_refresh cookie
    -> SHA-256(raw cookie)
    -> activePrincipalForRefresh(hash, now)
    -> check platform context when hq_id is null
    -> principal or HTTP 401
```

### سرویس‌های مستقیم

| وابستگی                  | کار در این کلاس                      |
|--------------------------|--------------------------------------|
| AccessTokenService       | decode توکن؛ صفحه ۱۳                 |
| AccessSessionValidator   | تأیید نشست؛ صفحه ۱۴                  |
| PlatformContextValidator | تأیید کاربر بدون سازمان؛ صفحه ۱۴     |
| SessionRepository        | یافتن نشست با هش refresh             |
| Clock → LaravelClock     | زمان جاری برای تشخیص انقضا           |
| OpaqueToken::hash        | تابع کمکی SHA-256؛ سرویس تزریقی نیست |

EloquentSessionRepository::activePrincipalForRefresh با refresh\_token\_hash جست‌وجو می‌کند؛ نشست نباید لغوشده یا منقضی باشد و کاربر باید ACTIVE باشد. مقدار جاری must\_change\_password را نیز می‌خواند. این شاخه مستقیماً RedisSessionRegistry را بررسی نمی‌کند.

احراز هویت موفق در این میدلویر به معنی تأیید خروج همگانی نیست. مرحله بعد، رمز فعلی یا مدرک step-up معتبر می‌خواهد. همچنین origin.exact قبل از این کلاس روی مسیر ثبت شده است.

مبنای کد:

Modules/Identity/src/Presentation/Http/Middleware/AuthenticateLogoutAll.php  
Modules/Identity/src/Infrastructure/Repositories/EloquentSessionRepository.php

## خروج همگانی: درخواست و پاسخ کامل

فرض: Origin مجاز، refresh cookie معتبر، رمز فعلی درست و دو نشست لغونشده وجود دارد. Authorization ارسال نشده؛ هویت از کوکی به دست می‌آید. بدنه باید دقیقاً یکی از current\_password یا step\_up\_verification\_token را داشته باشد.

```
POST /api/v1/auth/logout-all HTTP/1.1
Host: api.example.test
Accept: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Origin: https://panel.example.test
Cookie: chabok_refresh=REFRESH_VALID
Content-Type: application/json

{
  "current_password": "CorrectPassword!123"
}
```

```
HTTP 200
Content-Type: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111

{
  "data": {
    "revoked_session_count": 2
  },
  "meta": {},
  "correlation_id": "11111111-1111-4111-8111-111111111111"
}
```

کنترلر علاوه بر JSON، Set-Cookie حذف chabok\_refresh را با Path=/api/v1/auth، Secure، HttpOnly، SameSite=Lax و تاریخ گذشته می‌فرستد. عدد ۲ فرضی است. شمارش کد بر پایه نشست‌های revoked\_at=null است؛ لزوماً همه آن‌ها از نظر زمان هنوز معتبر نیستند.

### بعد از میدلویر چه سرویس‌هایی پاسخ را می‌سازند؟

- LogoutAllRequest ساختار و «دقیقاً یک مدرک» را بررسی می‌کند.
- LogoutAllHandler داخل TransactionManager رمز ذخیره‌شده را از CredentialRepository می‌گیرد و password\_verify می‌کند؛ یا VerificationProofConsumer مدرک PASSWORD\_RESET تأییدشده، منقضی‌نشده و متعلق به کاربر را مصرف می‌کند.
- SessionLifecycle نشست‌های کاربر را در دیتابیس لغو و در SessionRegistry علامت‌گذاری می‌کند. AuditWriter رویداد AUTH\_LOGOUT\_ALL می‌نویسد.
- RefreshCookieFactory کوکی را پاک و ApiResponder پاسخ نهایی را بسته‌بندی می‌کند.

اگر هیچ‌یک از دو راه احراز هویت موفق نشود: 401 AUTHENTICATION\_REQUIRED. اگر رمز ارائه‌شده اشتباه باشد: 401 INVALID\_CREDENTIALS با message برابر Invalid credentials. خطای دوم از Handler است، نه میدلویر.

مبنای کد:

```
Modules/Identity/src/Application/UseCases/LogoutAll/LogoutAllHandler.php
Modules/Identity/src/Application/Services/SessionLifecycle.php
Modules/Foundation/src/Presentation/Http/RefreshCookieFactory.php
```

## ۱. IdempotentCommand: دوبار کلیک، یک بار اجرا

فرض کن کلاینت درخواست ساخت را می‌فرستد، پاسخ گم می‌شود و دوباره همان درخواست را می‌فرستد. این میدلویر کمک می‌کند فرمان موفق دوباره اجرا نشود و پاسخ ذخیره‌شده برگردد. `alias` همراه نام فرمان ثبت می‌شود؛ مثل `idempotent:iam.roles.create`.

- `Idempotency-Key` الزامی است؛ `strlen` باید بین ۱۶ و ۱۲۸ باشد. در PHP این سنجش طول بایت است و این کد `trim` انجام نمی‌دهد.
- `principal` از احراز هویت قبلی گرفته می‌شود.
- اثر انگشت از متد، `path` و محتوای خام درخواست ساخته می‌شود.
- `ExecuteldempotentCommandHandler` تصمیم می‌گیرد فرمان تازه اجرا شود یا پاسخ قبلی بازپخش شود.

```
fingerprint = SHA256(method + "|" + path + "|" + rawBody)
record lookup = actor_id + command_name + idempotency_key
```

### درخواست واقعی بدون کلید

```
POST /api/v1/iam/roles HTTP/1.1
Host: api.example.test
Accept: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Authorization: Bearer TOKEN_VALID
Content-Type: application/json
```

```
{
  "role_code": "junior_demo",
  "role_title": "Junior demo",
  "permission_codes": []
}
```

```
HTTP 422
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Content-Type: application/json

{
  "error_code": "VALIDATION_ERROR",
  "message": "The request is invalid.",
  "field_errors": {
    "Idempotency-Key": [
      "A 16 to 128 character idempotency key is required."
    ]
  },
  "details": {},
  "correlation_id": "11111111-1111-4111-8111-111111111111"
}
```

مبنای کد:

Modules/Foundation/src/Presentation/Http/Middleware/IdempotentCommand.php

## سرویس اجرای فرمان تکرارپذیر

### ExecutIdempotentCommandHandler، گام به گام

- ۱. TransactionManager یک تراکنش دیتابیس باز می‌کند.
- ۲. IdempotencyRepository ابتدا رکورد کاربر را lock می‌کند و سپس رکورد همان actor + command + key را می‌خواند و lock می‌کند.
- ۳. اگر رکورد موجود و fingerprint متفاوت باشد: 409 IDEMPOTENCY\_KEY\_REUSED. اگر state به COMPLETED نرسیده باشد: 409 CONFLICT.
- ۴. اگر رکورد کامل و یکسان باشد، status و body قبلی با replayed=true برمی‌گردند.
- ۵. اگر رکوردی نبود، UUID می‌سازد و رکورد IN\_PROGRESS با expires\_at یک روز بعد درج می‌کند.
- ۶. callback یعنی ادامه زنجیره/کنترلر را اجرا می‌کند. پاسخ HTTP با status حداقل 400 ذخیره دائمی نمی‌شود و رکورد حذف می‌گردد.
- ۷. پاسخ با status کمتر از 400 با state=COMPLETED ذخیره می‌شود.

| سرویس                               | توضیح ساده                                                                    |
|-------------------------------------|-------------------------------------------------------------------------------|
| EloquentIdempotencyRepository       | قفل روی users و جدول idempotency_records؛ درج، به‌روزرسانی، حذف               |
| LaravelTransactionManager           | اجرای DB::transaction با attempts=3؛ تلاش مجدد فقط مطابق قواعد تراکنش Laravel |
| Clock → LaravelClock                | DateImmutable مبنی بر now برای زمان ثبت و انقضا                               |
| IdentifierGenerator → UuidGenerator | ساخت UUID رکورد با Str::uuid                                                  |
| Command / Result                    | اشیای حمل ورودی و خروجی؛ خودشان سرویس یا API نیستند                           |

اگر callback exception پرتاب کند، تراکنش rollback می‌شود. اگر فقط پاسخ خطای JSON برگرداند، Handler رکورد idempotency را حذف می‌کند؛ به‌تنهایی تضمین نمی‌کند تمام تغییرات تجاری قبل از آن rollback شوند.

مبنای کد:

Modules/Foundation/src/Application/UseCases/ExecuteIdempotentCommand/ExecuteIdempotentCommandHandler.php

Modules/Foundation/src/Infrastructure/Repositories/EloquentIdempotencyRepository.php

Modules/Foundation/src/Infrastructure/Persistence/LaravelTransactionManager.php

## تست Idempotency: ساخت نقش، بار اول

فرض: مدیر سازمان با مجوز و محدوده مدیریت نقش وارد شده و role\_code زیر هنوز وجود ندارد. نبود X-Node-Id عمدی است؛ این مثال روی کلید تکرار تمرکز دارد. بدنه بدون مجوز جدید معتبر است چون permission\_codes فقط باید present و array باشد.

```
POST /api/v1/iam/roles HTTP/1.1
Host: api.example.test
Accept: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Authorization: Bearer TOKEN_VALID
Idempotency-Key: junior-role-create-0001
Content-Type: application/json

{
  "role_code": "junior_demo",
  "role_title": "Junior demo",
  "description": null,
  "permission_codes": [],
  "menu_keys": []
}
```

```
HTTP 201
Content-Type: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111

{
  "data": {
    "role_id": "66666666-6666-4666-8666-666666666666",
    "hq_id": "55555555-5555-4555-8555-555555555555",
    "role_code": "junior_demo",
    "role_title": "Junior demo",
    "description": null,
    "role_kind": "CUSTOM",
    "is_cloneable": true,
    "status": "ACTIVE",
    "permission_codes": [],
    "menu_keys": []
  },
  "meta": {},
  "correlation_id": "11111111-1111-4111-8111-111111111111"
}
```

شناسه role\_id فرضی است. بدنه واقعی از CreateRoleHandler و RoleReader می‌آید. status برابر 201 را AuthorizationController تعیین می‌کند. میدلویر این status و متن JSON را برای تکرار بعدی نگه می‌دارد.

مبنای کد:

```
Modules/Authorization/src/Presentation/Http/Requests/CreateRoleRequest.php
Modules/Authorization/src/Presentation/Http/Controllers/AuthorizationController.php
Modules/Authorization/src/Application/Services/RoleReader.php
```

## تست Idempotency: تکرار، تعارض و نکات دقیق

### بار دوم: همان کلید و دقیقاً همان بدنه

درخواست صفحه قبل را با همان کاربر، مسیر، متد، key و بایت‌های بدنه تکرار کن. پاسخ نهایی دوباره 201 و همان role\_id است؛ کنترلر ساخت دوباره اجرا نمی‌شود. اگر X-Correlation-ID عوض شود، correlation\_id پاسخ بازپخش شده به مقدار درخواست جدید تغییر می‌کند.

### بار سوم: همان کلید و بدنه متفاوت

در همان درخواست فقط role\_title را از Junior demo به Junior demo 2 تغییر بده. پاسخ نهایی:

```
HTTP 409
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Content-Type: application/json

{
  "error_code": "IDEMPOTENCY_KEY_REUSED",
  "message": "The idempotency key was used for a different request.",
  "field_errors": {},
  "details": {},
  "correlation_id": "11111111-1111-4111-8111-111111111111"
}
```

### مواردی که جونیورها معمولاً اشتباه می‌گیرند

- حتی تغییر فاصله‌های JSON یا ترتیب کلیدها می‌تواند fingerprint را تغییر دهد، چون rawBody هش می‌شود.
- query string و X-Node-Id داخل fingerprint نیستند. هنگام طراحی فرمان جدید به اثر این ورودی‌ها توجه کن.
- کلید جست‌وجو شامل actor است؛ یک کلید برای دو کاربر یک رکورد مشترک نیست. tenant جداگانه در شرط lookup نیست.
- expires\_at ثبت می‌شود، اما lockRecord در این کد تاریخ انقضا را فیلتر نمی‌کند. گذشت ۲۴ ساعت به‌تنهایی مجوز اجرای دوباره نیست.
- پاسخ باید JsonResponse باشد؛ در غیر این صورت LogicException می‌گیری. شرط ذخیره status < 400 است، نه فقط 2xx.
- در replay همه هدرهای پاسخ قبلی ذخیره و بازسازی نمی‌شوند؛ status و body ذخیره شده‌اند.
- میدلویرهای قبل از idempotent دوباره اجرا می‌شوند؛ مثلاً احراز هویت و Audit انتخاب شعبه ممکن است دوباره انجام شوند.

مبنای کد:

Modules/Foundation/src/Presentation/Http/Middleware/IdempotentCommand.php

Modules/Foundation/src/Infrastructure/Repositories/EloquentIdempotencyRepository.php

## throttle: کنترل تعداد درخواست‌های احراز هویت

این کلاس اختصاصی پروژه نیست؛ alias داخلی Laravel است. سیاست‌های نام‌دار در FoundationServiceProvider ثبت شده‌اند. کلید گروه‌بندی از SHA-256 ترکیب IP و subject ساخته می‌شود؛ subject با trim و حروف کوچک نرمال می‌شود.

| سیاست           | حد در دقیقه | گروه‌بندی                       |
|-----------------|-------------|---------------------------------|
| auth-login      | ۱۰          | IP + identifier                 |
| auth-refresh    | ۳۰          | IP                              |
| auth-otp-send   | ۵           | IP + identifier                 |
| auth-otp-verify | ۱۰          | IP + challenge_id               |
| auth-password   | ۱۰          | IP؛ مشترک برای activate و reset |

### درخواست تستی برای رسیدن به سقف ورود

```
POST /api/v1/auth/login HTTP/1.1
Host: api.example.test
Accept: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Content-Type: application/json

{
  "identifier": "rate-limit-demo",
  "password": "wrong"
}
```

در محیط تست جداگانه، همین درخواست را ۱۱ بار در یک بازه محدودیت و از یک IP بفرست. با فرض عبور از مراحل قبلی، درخواست یازدهم به کنترلر ورود نمی‌رسد:

```
HTTP 429
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Content-Type: application/json

{
  "error_code": "RATE_LIMITED",
  "message": "Too many requests.",
  "field_errors": {},
  "details": {},
  "correlation_id": "11111111-1111-4111-8111-111111111111"
}
```

Cache و RateLimiter سرویس‌های این لایه‌اند. config/cache.php به صورت پیش فرض database است و CACHE\_STORE می‌تواند آن را عوض کند؛ تست‌های Integration این پروژه Redis را پوشش می‌دهند. bootstrap گزینه throttleWithRedis را فعال نکرده است.

Renderer پروژه برای 429 یک پاسخ تازه می‌سازد و هدرهای exception را صریحاً کپی نمی‌کند؛ وجود Retry-After را در این قرارداد تضمین شده فرض نکن. اجازه عبور throttle نیز به معنی درست بودن رمز نیست.

مبنای کد:

Modules/Foundation/src/Infrastructure/Providers/FoundationServiceProvider.php  
config/cache.php  
tests/Integration/AuthenticationSecurityTest.php

## لایه خود Laravel: میدلویرهای عمومی

این‌ها در پوشه Middleware پروژه نوشته نشده‌اند؛ از Laravel می‌آیند. فهرست پیش فرض بر اساس Middleware.php نسخه 13.22.0 در composer.lock بررسی شده است. کتابخانه vendor در این workspace نصب نیست؛ بنابراین فهرست runtime با artisan اجرا نشده است.

| میدلویر                                 | وظیفه و مثال ساده                                                                            |
|-----------------------------------------|----------------------------------------------------------------------------------------------|
| <b>ValidatePathEncoding</b>             | قالب UTF-8 مسیر را بررسی می‌کند؛ مسیر خراب ممکن است پیش از گروه api رد شود.                  |
| <b>InvokeDeferredCallbacks</b>          | کارهای defer شده واجد شرایط را هنگام پایان چرخه درخواست اجرا می‌کند؛ پاسخ تجاری مخصوص ندارد. |
| <b>TrustProxies</b>                     | مشخص می‌کند کدام هدر پراکسی برای IP/HTTPS قابل اعتماد است؛ به پیکربندی استقرار وابسته است.   |
| <b>HandleCors</b>                       | با CorsService و config/cors.php پاسخ مرورگر و preflight را مدیریت می‌کند.                   |
| <b>PreventRequestsDuringMaintenance</b> | در حالت maintenance درخواست‌های غیرمستثنا را متوقف می‌کند.                                   |
| <b>ValidatePostSize</b>                 | Content-Length را با سقف post_max_size مقایسه می‌کند.                                        |
| <b>TrimStrings</b>                      | فاصله رشته‌های ورودی را پاک می‌کند؛ password و current_password جزو استثنای پیش فرض اند.     |
| <b>ConvertEmptyStringsToNull</b>        | رشته خالی را در ورودی پردازش شده به null تبدیل می‌کند.                                       |
| <b>SubstituteBindings</b>               | در گروه api، پارامتر مسیر را در صورت تعریف binding به مدل/مقدار متناظر تبدیل می‌کند.         |

الگو: بدنه {"role\_title": " Demo ", "description"} در input پردازش شده می‌تواند title=Demo و description=null شود. rawBody برای fingerprint همچنان متن خام است. پاسخ موفق را کنترلر تعیین می‌کند؛ این تبدیل‌ها به تنهایی JSON جدیدی به کلاینت نمی‌دهند.

web middleware ها مثل StartSession و حفاظت CSRF به این گروه api اضافه نشده‌اند؛ وجود user\_sessions در دیتابیس، همان session معمول وب Laravel نیست. middleware آزمایشی ForcedPrincipal فقط در تست‌ها است و هویت ساختگی می‌گذارد.

مبنای کد:

bootstrap/app.php  
composer.lock

منبع رسمی: فهرست Middleware در Laravel 13.22.0 · رفتار TrimStrings در شاخه x.13

## نکته مهم CORS: پاسخ واقعی preflight

HandleCors سراسری قبل از CorrelationId و RequireSecureTransport گروه api قرار می‌گیرد. config/cors.php مسیر `*/api` را پوشش می‌دهد. طبق پیاده‌سازی رسمی Laravel، اگر درخواست preflight باشد، HandleCors پاسخ را همان‌جا می‌دهد و next را صدا نمی‌زند.

```
OPTIONS /api/v1/auth/refresh HTTP/1.1
Host: api.example.test
Accept: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Origin: https://panel.example.test
Access-Control-Request-Method: POST
Access-Control-Request-Headers: content-type,x-correlation-id
```

```
Expected preflight response (allowed origin):
HTTP 204
Access-Control-Allow-Origin: https://panel.example.test
Access-Control-Allow-Credentials: true
Access-Control-Allow-Methods: GET, POST, PATCH, PUT, DELETE, OPTIONS

<empty body>
```

این مثال هدرهای کلیدی را نشان می‌دهد؛ جزئیات Vary و قالب سایر هدرها وابسته به CorsService است. نمی‌توان وجود correlation\_id یا اجرای بررسی HTTPS سفارشی را برای این پاسخ تضمین کرد؛ درخواست هنوز به آن میدلویرها نرسیده است.

### مبدأ نامجاز چه می‌شود؟

403 صریح ORIGIN\_NOT\_ALLOWED رفتار شاخه preflight در CredentialedCors سفارشی است، در صورتی که درخواست به آن برسد. برای preflight معمول `*/api`، لایه سراسری می‌تواند زودتر پاسخ بدهد و هدر اجازه مبدأ را ندهد؛ بنابراین همه OPTIONS‌های نامجاز را با قطعیت 403 معرفی نکن.

### خطا پیش از CorrelationId

اگر درخواست قبل از گروه api خطا بخورد، attributes شناسه پیگیری هنوز تنظیم نشده است. Renderer برای برخی exception‌های عمومی مانند اندازه بیش از حد یا maintenance نگاشت اختصاصی ندارد و در صورت رسیدن به default، پاسخ 500 INTERNAL\_SERVER\_ERROR با correlation\_id خالی می‌سازد. جزئیات نهایی به نقطه توقف، وب‌سرور و مسیر رندر بستگی دارد.

منبع رسمی: HandleCors در شاخه x.13؛ رفتار این لایه با کد محلی config/cors.php و bootstrap/app.php تطبیق داده شده است.

مبنای کد:

config/cors.php  
Modules/Foundation/src/Presentation/Http/ApiExceptionRenderer.php

## یک درخواست کامل: از توکن تا لیست نشست‌ها

فرض: کاربر یک نشست دارد، `mustChangePassword=false` است، شعبه انتخاب‌شده متعلق به سازمان اوست و مجوز `node_context.switch` با `scope` مناسب دارد. تاریخ‌ها نمونه‌اند.

```
GET /api/v1/me/sessions HTTP/1.1
Host: api.example.test
Accept: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111
Authorization: Bearer TOKEN_VALID
X-Node-Id: 22222222-2222-4222-8222-222222222222
```

```
HTTP 200
Content-Type: application/json
X-Correlation-ID: 11111111-1111-4111-8111-111111111111

{
  "data": [
    {
      "session_id": "44444444-4444-4444-8444-444444444444",
      "device_id": "junior-laptop",
      "device_name": "Test browser",
      "issued_at": "2026-09-20T08:00:00+00:00",
      "expires_at": "2026-10-20T08:00:00+00:00",
      "last_seen_at": "2026-09-20T08:00:00+00:00",
      "revoked": false
    }
  ],
  "meta": {},
  "correlation_id": "11111111-1111-4111-8111-111111111111"
}
```

مراحل: شماره پیگیری → HTTPS → خواندن شعبه → احراز توکن و نشست → تأیید دسترسی شعبه و Audit → وضعیت رمز → `.IdentityController::sessions` → `ListSessionsHandler` → `SessionRepository` → `ApiResponder`

`ListSessionsHandler` لیست همه نشست‌های کاربر را می‌گیرد و `revoked` را از `revoked_at` می‌سازد؛ لیست الزاماً فقط نشست‌های فعال نیست. GET ساختار داده را تغییر نمی‌دهد، اما `node.access` می‌تواند Audit انتخاب شعبه بنویسد.

مبنای کد:

Modules/Identity/src/Application/UseCases/ListSessions/ListSessionsHandler.php  
Modules/Identity/src/Infrastructure/Repositories/EloquentSessionRepository.php

## تست دستی با curl، بدون حدس زدن ورودی‌ها

این دستورها نمونه قابل ویرایش‌اند و در این بررسی اجرا نشده‌اند. BASE را آدرس واقعی محیط تست قرار بده؛ توکن را از پاسخ ورود همان محیط بگیر. Origin باید عین مقدار فهرست مجاز باشد.

```
BASE="https://api.example.test"
ORIGIN="https://panel.example.test"
CID="11111111-1111-4111-8111-111111111111"
TOKEN="REPLACE_WITH_REAL_ACCESS_TOKEN"
```

### ۱. گرفتن توکن و کوکی در محیط تست

```
curl -i -c /tmp/chabok-test-cookies.txt \
"$BASE/api/v1/auth/login" \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-H "Origin: $ORIGIN" -H "X-Correlation-ID: $CID" \
--data-raw '{"identifier":"TEST_USER","password":"TEST_PASSWORD"}'
```

اگر ورود موفق باشد data.access\_token، HTTP 200 و Set-Cookie دریافت می‌کنی؛ مقدار refresh\_token در JSON عمومی قرار نمی‌گیرد. TOKEN را با data.access\_token جایگزین کن. پاسخ ورود context و مشخصات کاربر نیز دارد.

### ۲. مشاهده نشست‌ها و ۳. ساخت خطای هدر

```
curl -i "$BASE/api/v1/me/sessions" \
-H "Accept: application/json" \
-H "Authorization: Bearer $TOKEN" \
-H "X-Correlation-ID: $CID"

curl -i "$BASE/api/v1/me/sessions" \
-H "Authorization: Bearer $TOKEN" \
-H "X-Correlation-ID: $CID" \
-H "X-Node-Id: invalid-node"
```

تست ۲ با فرض وضعیت رمز مناسب، پاسخ ساختار صفحه ۲۸ را دارد. تست ۳ قبل از احراز توکن در NodeContext با 422 متوقف می‌شود. برای تست دسترسی واقعی، به جای invalid-node یک UUID واقعی ولی خارج از محدوده کاربر بگذار.

مبنای کد:

Modules/Identity/src/Application/Services/SessionIssuer.php  
Modules/Identity/src/Presentation/Http/Requests/LoginRequest.php

## چطور تست کنیم که دقیقاً کدام لایه شکست خورده؟

هر بار فقط یک چیز را خراب کن و بقیه پیش شرطها را معتبر نگه دار. مثلاً برای تست `node.access` باید UUID درست، توکن معتبر، سازمان معلوم و دیتابیس آماده باشد؛ وگرنه قبل از آن در لایه دیگری متوقف می شوی.

| آزمایش                               | نتیجه مورد انتظار                              |
|--------------------------------------|------------------------------------------------|
| Correlation ID نامعتبر               | correlation_id در خراب در VALIDATION_ERROR 422 |
| HTTP و استثنای غیرفعال               | Secure transport is required ؛FORBIDDEN 403    |
| logout بدون Origin                   | MISSING با metric ؛ORIGIN_NOT_ALLOWED 403      |
| X-Node-Id نامعتبر                    | VALIDATION_ERROR 422                           |
| GET محافظت شده بدون Bearer           | AUTHENTICATION_REQUIRED 401                    |
| توکن با نشست لغوشده / کاربر غیرفعال  | AUTHENTICATION_REQUIRED 401                    |
| شعبه متعلق به سازمان دیگر            | TENANT_ACCESS_DENIED 403                       |
| شعبه هم سازمان ولی بدون مجوز switch  | PERMISSION_DENIED 403                          |
| مجاز موجود ولی scope نامناسب         | SCOPE_ACCESS_DENIED 403                        |
| mustChangePassword=true روی sessions | PASSWORD_CHANGE_REQUIRED 403                   |
| logout-all با refresh و مدرک درست    | 200 و تعداد نشست های لغوشده                    |
| کلید idempotency کوتاه               | VALIDATION_ERROR 422                           |
| کلید تکراری و rawBody یکسان          | status و body قبلی؛ correlation جدید           |
| کلید تکراری و rawBody متفاوت         | IDEMPOTENCY_KEY_REUSED 409                     |
| یازدهمین login در بازه محدودیت       | RATE_LIMITED 429                               |

### ترتیب عیب یابی

اول status و error\_code را بخوان. بعد correlation\_id را در لاگ پیدا کن. سپس هدرها، پیکربندی Origin/HTTPS، نشست و وضعیت کاربر، permission و scope را بررسی کن. خطای تجاری بعد از عبور از میدلویرها هم ممکن است؛ هر 403 الزاماً از node.access نیست.

CORS را فقط با curl تأیید نکن؛ curl سیاست مرورگر را اجرا نمی کند. اگر curl پاسخ می گیرد و مرورگر نمی خواند، Origin، هدرهای پاسخ، کوکی و preflight را در Network مرورگر بررسی کن.

## تست‌های موجود و مرز اعتبار این سند

| فایل تست                                                      | چه چیزی را پوشش می‌دهد                                                                            |
|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <b>Modules/Foundation/tests/Feature/ApiFoundationTest.php</b> | CORS, envelope, correlation, HTTPS, Origin هدر شعبه، تغییر رمز اجباری، JWT و پوشاندن اطلاعات حساس |
| <b>tests/Integration/AuthenticationSecurityTest.php</b>       | logout و logout-all، گردش refresh، رمز اجباری، کاربر پلتفرم، انتخاب شعبه و سقف درخواست            |
| <b>tests/Integration/AuthorizationIntegrationTest.php</b>     | اتصال adapterها، کش دسترسی، تفکیک tenant/permission/entitlement/scope و نقش پلتفرم                |
| <b>tests/Integration/PersistenceAndAdministrationTest.php</b> | بازپخش درخواست و تغییر fingerprint؛ rollback تجاری و لغو نشست                                     |
| <b>tests/Feature/ModuleRouteRegistrationTest.php</b>          | ثبت مسیرها و وجود میدلویرهای لازم روی عملیات مشخص                                                 |

### این سند چگونه راستی‌آزمایی شده؟

فایل‌های میدلویر، bootstrap، Routes، ServiceProvider، سرویس‌های فراخوانی‌شده، کنترلرهای نمونه و تست‌های مرتبط خوانده شده‌اند. نمونه‌های موفق بر اساس پیش‌شرط مشخص و ساختار واقعی ApiResponseدر نوشته شده‌اند. آزمون PHP/Integration و درخواست روی محیط زنده در این کار اجرا نشده‌اند؛ vendor در workspace موجود نیست.

### رفتار فعلی را با انتظار محصول قاطی نکن

- NodeContext به‌تنهایی مجوز نمی‌دهد و ValidateNodeAccess بدون هدر شعبه الزام ایجاد نمی‌کند.
  - اجازه CORS با احراز هویت فرق دارد؛ کاربر curl می‌تواند Origin بفرستد، پس این هدر هویت کاربر نیست.
  - محدودیت ورود روی همه APIها اعمال نشده؛ فقط مسیرهای throttleدار.
  - انقضای رکورد idempotency در lookup فعلی بررسی نمی‌شود.
  - ثبت Audit انتخاب شعبه قبل از کنترلر انجام می‌شود؛ این رویداد نشان نمی‌دهد عملیات نهایی حتماً موفق شده است.
- برای یادگیری سریع، ابتدا مسیر صفحه ۲۸ را با دست دنبال کن؛ بعد یک شرط را خراب کن و جدول صفحه ۳۰ را کنار پاسخ بگذار. این کار تفاوت «قالب ورودی»، «هویت»، «اجازه» و «اجرای فرمان» را روشن می‌کند.

## نقشه فایل‌ها برای ادامه مطالعه

همه مسیرها نسبت به ریشه ChabokPro هستند. برای پیدا کردن هر کلاس، نام آن را در جست‌وجوی IDE وارد کن. این فهرست مسیرهای اصلی را نشان می‌دهد؛ مسیرهای وابستگی‌های جزئی در صفحات مربوط آمده‌اند.

مبنای کد:

```
bootstrap/app.php
config/chabok.php
config/cors.php
config/cache.php
Modules/Foundation/src/Presentation/Http/Middleware/*.php
Modules/Identity/src/Presentation/Http/Middleware/AuthenticateLogoutAll.php
Modules/Foundation/src/Infrastructure/Providers/FoundationServiceProvider.php
Modules/Identity/src/Infrastructure/Providers/IdentityServiceProvider.php
Modules/Authorization/src/Infrastructure/Providers/AuthorizationServiceProvider.php
Modules/Identity/src/Infrastructure/Security/DatabaseAccessSessionValidator.php
Modules/Identity/src/Infrastructure/Security/RedisSessionRegistry.php
Modules/Foundation/src/Infrastructure/Security/FirebaseAccessTokenService.php
Modules/Authorization/src/Application/Services/AuthorizationGuard.php
Modules/Authorization/src/Application/UseCases/ResolveContext/ResolveContextHandler.php
Modules/Foundation/src/Application/ScopedAccess.php
Modules/Authorization/src/Infrastructure/Repositories/EloquentAuthorizationRepository.php
Modules/Organization/src/Infrastructure/Adapters/EloquentScopeTopology.php
Modules/Audit/src/Infrastructure/Persistence/MySQLAuditWriter.php
Modules/Foundation/src/Application/UseCases/ExecuteIdempotentCommand/
Modules/Foundation/src/Infrastructure/Repositories/EloquentIdempotencyRepository.php
Modules/Identity/src/Infrastructure/Repositories/EloquentSessionRepository.php
Modules/Foundation/src/Presentation/Http/ApiResponse.php
Modules/Foundation/src/Presentation/Http/ApiExceptionRenderer.php
```

### منابع رسمی تکمیلی Laravel

فهرست پیش‌فرض میدلویرها از نسخه 13.22.0 خوانده شده؛ توضیح جزئی `HandleCors`، `TrimStrings` و `ValidatePathEncoding` از شاخه رسمی 13.x است و ممکن است با تغییرات بعدی آن شاخه تفاوت پیدا کند.

- Middleware configuration - Laravel 13.22.0
- HandleCors - Laravel 13.x
- TrimStrings - Laravel 13.x
- ValidatePathEncoding - Laravel 13.x